

提示词结构

那么，前面介绍的提示词工程的通用原则，该如何运用到我们的实验上来呢？在这套操作系统实验中，提示词该怎么组织？用什么格式？每个部分写什么？

- 参考资料：[基于大语言模型（LLM）的生成式文件系统](#)

由此，我们设计了一个标准的四段式框架，所有实验统一使用。你可以根据具体任务调整内容，但整体骨架保持一致。

四段式框架总览

[PROMPT]

做什么、改哪里、怎么改

[RELY]

可以依赖什么——宏定义、数据结构、已有函数

[GUARANTEE]

必须实现或修改什么——需要交付的函数列表

[SPECIFICATION]

每个函数的详细行为规格

四个部分各有分工，缺一不可。下面逐个展开。

[PROMPT]：任务说明

[PROMPT]回答的是“你要做什么”。它包含三个字段：

任务：说清楚目标文件和模块的具体功能。如果涉及多个文件，每个文件负责什么，要在这里交代。不要让AI自己猜该改哪个文件。

操作要求：约束AI的行为方式。比较重要的一条是“必须在项目中修改实际文件”——如果不强调这一点，AI很可能只输出代码片段而不真正改文件。此外还可以说清楚：保留现有代码，只在标记位置补全，不要删除既有逻辑。

输出要求：告诉AI使用下面[RELY]、[GUARANTEE]、[SPECIFICATION]中提供的信息，以工程真实代码为准，不要引入无关功能。

以物理内存初始化任务为例：

****任务**：**修改pmm.c文件，实现物理内存探测与Page描述符初始化功能。

****操作要求**：**你必须在项目中修改实际文件，保留现有代码，只在标记位置补全。

****输出要求**：**直接进行文件操作，使用下面[RELY]、[GUARANTEE]、[SPECIFICATION]部分的信息。

[RELY]：上下文信息

[RELY]提供AI完成任务所需的最小可信上下文——宏定义、数据结构、可调用的函数、相关全局变量。

关键词是"最小"和"可信"。

"最小"意味着不要把整个工程的头文件都贴进来，只放和当前任务直接相关的。无关信息太多，AI反而容易分心。

"可信"意味着这些信息必须和工程代码一致。去代码里把真实的定义复制过来，不要凭记忆自己写一个"差不多"的版本。如果工程里的宏叫 PG_SIZE ，你写成 PAGE_SIZE ，AI生成的代码就会编译不过。

如果某个数据结构的布局对任务很关键（比如 struct Page 描述符数组的地址范围和对应的物理地址），应该把相关字段和地址关系明确列出来，而不是只给一个结构体名。

[GUARANTEE]：接口清单

[GUARANTEE]列出这次任务必须实现或修改的所有函数签名。它是一份"交付清单"：这些函数必须在最终代码中出现，签名必须和声明一致。

对于需要实现的新函数，明确指出函数签名和预期的接口行为；对于需要修改的现有函数，应说明修改的目标和预期效果。

除了主接口，通常还需要说明是否允许新增辅助函数。如果允许，加上约束：辅助函数应该是static的，只在本模块内部使用，不应该改变公开接口。

一个[GUARANTEE]块中的接口应该属于同一个功能模块。比如物理内存初始化相关的 `page_init`、`get_memory_base`、`get_memory_size`、`init_memmap` 和 `pa2page` 放在一起；它们可以跨文件，但应该归属同一个模块目标。

[SPECIFICATION]: 行为规格

这是提示词最核心的部分。对每个主要函数，都要用统一的格式描述它的行为：

Pre-Condition：调用这个函数前，系统应该处于什么状态？参数有什么约束？

Post-Condition：函数执行完成后，系统应该处于什么状态？这里描述的是函数执行后的总体状态，包括数据结构、资源、全局状态等方面必须满足的条件。

Case 1、Case 2.....：在Post-Condition下面，按照不同的执行情况分别描述函数的具体行为。每个Case都用一段话说明在什么条件下触发、需要执行什么操作，以及执行完成后需要满足什么状态。对操作系统代码来说，一个函数可能存在多种不同的执行路径，例如正常执行、参数异常、资源不足或特殊系统状态等。将这些情况分别列出，可以更准确地描述函数的完整行为。

Requirements：特殊的时序要求、并发约束、性能要求，或者其他特殊需求。

其中，**Case和Requirements都是可选的**，是否使用应根据具体函数的实际情况决定。如果函数只有单一执行路径或不存在需要单独描述的特殊执行路径，不需要设置Case；如果函数没有额外的特殊要求，不需要设置Requirements。

还需要注意的是，各个字段描述的是**函数需要实现什么行为**，而不是**具体如何实现**。因此，应尽量避免堆积大量具体的函数调用、变量名和常量。规格应该足够抽象、清晰，不拘泥于过于具体的实现细节，从而更适合作为需求和设计说明。

提示词的粒度

一个提示词该覆盖多大范围？

不是“一个函数一个提示词”，也不是“一个章节一个提示词”。比较合适的粒度是：一个可以独立实现、独立验证，并且能够完整完成一项功能的任务，一个提示词。

什么叫“能够完整完成一项功能”？简单来说，就是这一组函数共同负责一件事情，它们在正常执行和异常情况下涉及的资源、状态和处理逻辑都是相互关联的。如果强行拆开，AI很容易只理解其中一个函数，而忽略整个功能涉及的状态变化。

比如，物理内存初始化涉及内存探测、地址转换、Page 描述符布局、已用页面保留和空闲区注册等多个环节，它们彼此紧密依赖，拆成多个提示词反而会让每个提示词都缺少必要的上下文。合在一起，AI才能理解物理内存上界、OpenSBI 和内核镜像占用空间、描述符数组占用空间，以及剩余完整页面之间的关系。

标准模板

下面给出两种场景的模板。实际使用时，可以将尖括号中的占位符替换为具体内容，也可以按自己的想法进行细微调整。

单函数模板

[PROMPT]

****任务****: 修改文件 `<file_path>` (可以是多个文件), 实现或修改 `<function_name>` 函数。

****操作要求****: 你必须在项目中修改实际文件, 而不是仅展示代码。完成函数的实现或修改, 包括但不

****输出要求****: 直接进行文件操作。保留文件中的现有代码。实现或修改的代码应插入或更新到相应的

[RELY]

// 宏定义

`<relevant_macros>`

// 数据结构

`<relevant_structs>`

// 可用函数

`<available_functions>`

[GUARANTEE]

必须实现或修改的接口:

```c

`<function_signature>`

```

你可以根据需要添加:

- 辅助函数 (建议设为`static`)
- 内部使用的宏定义
- 模块内部的静态变量

[SPECIFICATION]

`<function_name>`

****Pre-Condition****:

- `<前置条件>`

****Post-Condition****:

- `<后置条件>`

****Case 1****:

- `<一段话描述: 触发条件、执行行为和最终状态>`

****Case 2****:

- <一段话描述：触发条件、执行行为和最终状态>

****Case 3**:**

- <一段话描述：触发条件、执行行为和最终状态>

****Requirements**:**

- <额外的系统约束>

多函数模板

[PROMPT]

****任务****: 修改文件 `<file_path>` (可以是多个文件), 实现或修改 `<module_name>` 模块 (可以将

****操作要求****: 你必须在项目中修改实际文件, 而不是仅展示代码。完成功能模块的实现或修改, 包括

****输出要求****: 直接进行文件操作。保留文件中的现有代码。实现或修改的代码应插入或更新到相应的

[RELY]

`<context>`

[GUARANTEE]

必须实现或修改的主要接口:

```c

`<function_signatures>`

```

你可以根据需要添加:

- 辅助函数 (建议设为`static`)
- 内部使用的宏定义
- 模块内部的静态变量

[SPECIFICATION]

`<func1>`

`<详细规格>`

`<func2>`

`<详细规格>`

`<func3>`

`<详细规格>`



写好之后的检查

提示词写完了, 提交之前过一遍这几个问题:

- 目标文件和目标模块写清楚了吗?
- 说了"必须修改真实文件"吗?

- 所有必须实现或修改的接口都列出来了吗？
- [RELY]里的信息和工程代码一致吗？
- 允许和禁止修改的范围写清楚了吗？
- 每个Case的描述是否明确？
- 关键的系统约束覆盖到了吗？

如果有哪一条答不上来，说明提示词还不够完整，还得返回再补充。

下一步：实际操作

框架有了，模板也有了。但具体到你的操作系统实验任务，该怎么用这套框架？从拿到任务到生成代码再到验证通过，完整的工作流程是什么？

接下来的实践指南会告诉你：如何理解任务、如何梳理需求、如何编写提示词、如何验证代码，以及如何在遇到问题时调整策略。